Card Vision

1st Alessio Simoncini
dept. of Information Engineering
University of Pisa
a.simoncini9@studenti.unipi.it

2nd Lorenzo Vezzani
dept. of Information Engineering
University of Pisa
l.vezzani1@studenti.unipi.it

***Abstract*—This paper presents the development, training, and evaluation of a YOLO-based object detection model for recognizing standard playing cards, such as those used in Poker and other card games. Multiple YOLO model variants were trained and evaluated in terms of detection accuracy, inference time, and model size in order to analyze the trade-off between computational efficiency and recognition performance. In addition, the work addresses the Same-Card Recognition problem, which arises from the structure of the available datasets, and proposes a practical methodology to infer whether two detected card corners belong to the same physical card. Based on the experimental evaluation, the most suitable model was selected and integrated into a prototype Poker game assistant, demonstrating the applicability of the proposed approach in a real-world scenario.**

***Index Terms*—machine learning, object detection, yolo, game assistant, montecarlo**

I. Introduction

Automatic playing card recognition is a computer vision task with applications in a wide range of domains, including game analysis, augmented reality, automated scoring systems, and decision support tools for card games such as Poker. The availability of accurate and efficient object detection models enables the development of intelligent systems capable of understanding the state of a game directly from camera images, without requiring specialized hardware.

Advances in deep learning, and in particular the YOLO family of object detection models, have significantly improved the accuracy and speed of real-time object recognition. These characteristics make YOLO models well suited for playing card recognition, where both precise localization and low inference latency are essential for practical deployment.

This work focuses on the use of YOLOv11 models for playing card recognition. Different model variants are trained and evaluated in terms of detection accuracy, inference time, and model size in order to analyze the trade-off between computational cost and performance. The comparison provides guidance for selecting the most appropriate model according to the requirements of different deployment scenarios.

In addition, this work addresses the Same-Card Recognition problem, which arises when individual card corners are annotated but do not explicitly indicate whether multiple detections belong to the same physical card. A geometric approach based on bounding box distance and areas *Distance over Area* (*DoA*) metric is proposed to infer this relationship and is experimentally evaluated.

Finally, the selected model is integrated into a prototype Poker hand analysis system, demonstrating the feasibility of the proposed approach in a realistic application scenario.

II. Related Works

A. Playing Card Detection

Many works have applied and trained YOLO-based detectors to the field of playing cards.

Xu et al. [1] trained a YOLOv3m model to detect playing cards using synthetically generated scenes with cards of varying size and rotation; they developed mobile system to provide user with suggestion for the opening bid in Bridge.

Kostadinov [2] trained multiple YOLOv8m variants on various custom, real and synthetic datasets, comparing their results and deploying the best-performing model in a live demo. Multiple other university projects can be found on the argument, but none that compared and analyzed multiple variants of a YOLO model.

B. Card Games analysis and assistants

Several prior works have used card recognition as a base for a gameplay assistant application.

At Tooploox [3] they trained a YOLOv2-tiny model on a small custom dataset and developed an Android mobile application able to recognize a player's hand.

Singh [4] developed a real-time webcam system that analyzes at every frame the composition of the card in shot, classifying the hand according to poker definitions (Flush, Straight, etc.).

Fay and Wong [5] developed a system to evaluate the strength of a poker hand via Monte Carlo simulation, without developing an underlying card recognition system.

III. Backgrounds

A. Use Case Scenarios

The proposed Intelligent System is designed to support automatic playing card recognition and analysis through computer vision techniques. The following use-case scenarios illustrate how the system can be employed in practice.

Poker assistant — A player points a camera toward the playing table while a poker game is in progress. The system acquires the image, detects and recognizes the cards and forwards the recognized cards to the poker analysis module. The module determines the current hand configuration and

gives the player relevant game information, such as hand evaluation and suggested actions.

Game digitization — A tournament organizer relies on the Intelligent System to continuously capture the game state during a card game, by recognizing the cards on the table. The recognized information is stored to automatically generate a digital record of the match, enabling replay, archival, or post-game review.

Statistical analysis, Decision support — The recognized cards are collected across multiple games and provided as input to higher-level analysis modules, to be used for further purposes. A game analyst may use this data to compute game statistics, identify recurring game patterns, and analyze player behaviors and strategies. A researcher can use the gathered game data as a foundation for training more complex, machine learning-based decision-support systems.

B. Machine Learning Task Definition

The main Machine Learning task addressed in this work is object detection. Given an input image containing one or more playing cards, the objective is to identify the position of each card through bounding box localization and assign the corresponding card class.

The functional requirements of the object detection component (and relative module) for our application are:

- Detection of all visible playing cards in an input image
- Classification of each detected card among the 52 possible card classes
- Sufficiently low inference time to enable real-time applications.

Moreover, the system must address the Same-Card Recognition task. Since two detected card corners may belong to the same physical card, the system must determine whether two detections correspond to the same card instance, to correctly keep track of the number of cards currently in-game.

C. Intelligent System Architecture

The proposed Intelligent System is composed of multiple components organized in a sequential pipeline; the overall architecture can be schematized as follows:

- **Image acquisition module** — receives images from a camera or an external image source
- **Card detection module** — based on a YOLO object detection model, identifies playing cards and predicts their corresponding bounding boxes and classes
- **Same-Card Recognition module** — analyzes pairs of detections belonging to the same class in order to determine whether they correspond to the same physical card
- **Poker analysis module** — processes the recognized cards and derives information about the current poker hand
- **User interface/output module** — provides the final information to the user

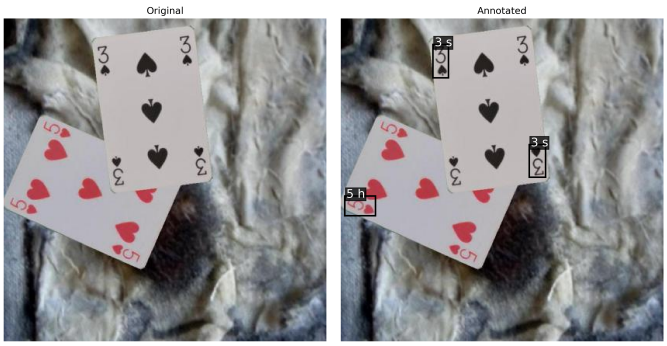


Fig. 1. Example image from Kaggle dataset

IV. Datasets

Two different dataset have been chosen because of the mainly synthetic nature of the datasets found: from preliminary analysis it resulted that the models trained on just one of the two dataset would have excellent performance on their respective test set but very poor generalization capabilities over images from other sources. Given this, two datasets have been merged to introduce more variability in the images and improve model generalization.

In both datasets 52 classes are defined, corresponding to all possible combinations of card rank (*A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K*) and card suit (*hearts, diamonds, clubs, spades*). Bounding boxes are annotated on the top-left and bottom-right corner of the card, enclosing the rank and suit symbol that identify the card, as can be seen in Fig. 1 and Fig 2.

A. Kaggle dataset

The first dataset can be found on Kaggle [7]. This has been constructed by taking 20-30 second videos of all 52 cards under variable light temperature and brightness; the images were processed with open-cv, where they were placed over random backgrounds. Image dimensions are 416x416 pixels. The dataset is provided split into train/valid/test (70/20/10 split). An example image from the Kaggle dataset is shown in Fig. 1.

Some statistics about the dataset:

- Total number of images: 20,000
- Total number of bounding boxes: 75,750
- Average number of bounding boxes per image: 3.79 ($\sigma = 0.44$)
- Average number of categories per image: 2.45 ($\sigma = 0.54$)
- Average number of bounding boxes per category: $1,456.73$ ($\sigma = 51.18$)
- Category with the highest number of bounding boxes: *Three of Spades* (1,588)
- Category with the lowest number of bounding boxes: *Queen of Clubs* (1,358)

B. Roboflow dataset

The second dataset can be found on Roboflow [6] This has been constructed in a completely synthetical way, by

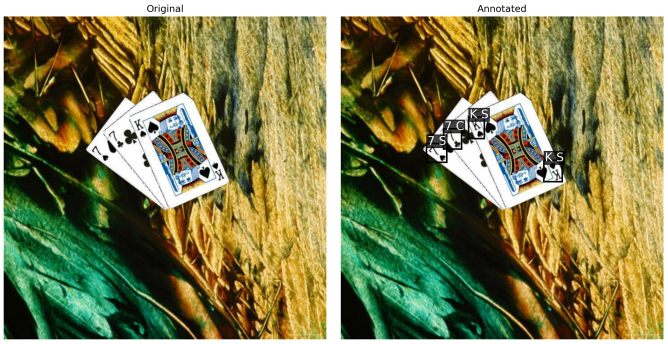


Fig. 2. Example image from Roboflow dataset

generating images of card over a random background. Image dimensions are 720x720 pixels. The dataset is provided split into train/valid/test (70/20/10 split). An example image from the Roboflow dataset is shown in Fig. 2.

Some statistics about the dataset:

- Total number of images: 10,100
- Total number of bounding boxes: 40,400
- Average number of bounding boxes per image: 4.00 ($\sigma = 0.00$)
- Average number of categories per image: 2.95 ($\sigma = 0.23$)
- Average number of bounding boxes per category: 776.92 ($\sigma = 31.67$)
- Category with the highest number of bounding boxes: *Five of Clubs* (854)
- Category with the lowest number of bounding boxes: *Six of Hearts* (712)

C. Data augmentation

Data augmentation was introduced to improve the robustness and generalization capabilities of the model; this was deemed necessary beacuse of the synthetic nature of the images, which are all perfectly in shot, on focus and illuminated.

Augmentation was automatically applied during the YOLO training phase thanks the augmentation pipeline provided by the framework. The implemented augmentations include photometric transformation (brightness and saturation), geometric transformation (scaling, translation, shearing and perspective changes) and blur transformation. The complete set of augmentations and corresponding parameters are documented in detail in the training notebook.

V. YOLO MODEL COMPARISION

A. Experimental Setup

The experiments were conducted on a high-performance computing system with the following configuration:

- **CPU**: Dual Intel Xeon Platinum 8480+ (112 cores total, NUMA architecture with 8 nodes)
- **GPU**: NVIDIA A100-SXM4-80GB

All experiments were performed under strictly identical conditions to ensure a fair comparison between models.

Performance was assessed using standard object detection metrics:

- **mAP@50**: measures detection accuracy under a fixed IoU threshold, providing an intuitive evaluation of correct detections.
- **mAP@50-95**: evaluates localization quality across multiple IoU thresholds and is the standard COCO benchmark metric.

Other factors used for the final selection are:

- **Inference time**: measures the time required to process a single image, determining the suitability of the model for real-time applications.
- **Model dimension**: represents the storage and memory requirements of the trained model, affecting deployment on devices with limited computational resources.

The evaluated models are YOLOv11-Nano, YOLOv11-Small, YOLOv11-Medium, and YOLOv11-Large, enabling an analysis of the trade-off between computational complexity and detection performance. All models were initialized from the corresponding pre-trained weights and fine-tuned on the constructed dataset.

B. Cross validation

We performed cross-validation to evaluate our models: a 5-fold cross-validation strategy was adopted. The entire dataset was divided into five mutually exclusive subsets (folds), each containing approximately the same number of samples.

At each iteration, one fold was used as the test set, while the remaining four folds were combined to form the development set. From this development set, 80% of the samples were used for training and the remaining 20% were used for validation. The procedure was repeated five times, so that each fold served exactly once as the test set.

This evaluation protocol ensures that every sample is used exactly once for testing and four times for training/validation, providing a reliable estimate of the model's generalization performance while reducing the dependence on a single train–test split.

It is worth noting that a fully nested cross-validation approach would represent a more rigorous evaluation protocol. In a nested cross-validation scheme, the outer loop would be used exclusively for estimating the generalization performance, while an inner loop would be used for model selection and hyperparameter tuning.

The simplified approach adopted in this work combines model evaluation and selection within the same cross-validation framework. While this may introduce a limited optimistic bias in the performance estimation, the impact is expected to be small in this case due to the absence of extensive hyperparameter optimization.

The *mAP@50–95* results are presented in Table I.

C. Results analysis

The underlying playing card detection task is straightforward, as playing cards are well-defined rectangular white

TABLE I
CROSS-VALIDATION RESULTS (MAP@0.5:0.95) FOR THE EVALUATED YOLO MODELS.

Fold	YOLO-N	YOLO-S	YOLO-M	YOLO-L
1	0.8230	0.9093	0.9354	0.9506
2	0.8202	0.9198	0.9483	0.9477
3	0.8183	0.9177	0.9449	0.9482
4	0.8470	0.9125	0.9480	0.9470
5	0.7841	0.9186	0.9456	0.9496
Mean	0.8185	0.9156	0.9444	0.9486
Std. Dev.	0.0210	0.0044	0.0053	0.0015

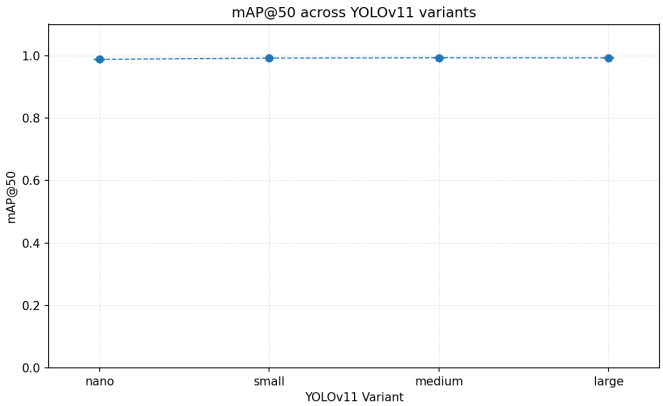


Fig. 3. mAP@50 across YOLOv11 variants

objects, and the target bounding boxes are consistently aligned with their corners; as a consequence model performance is very high starting from the Nano variant.

The *mAP@50* metric shows excellent results across all model architectures, as reported in Fig. 3. Even the YOLOv11-Nano model achieves a value of approximately 0.98, while the YOLOv11-Large variant reaches values close to 0.99. This indicates that all models are highly capable of correctly detecting playing cards when considering a single IoU threshold of 0.5.

A larger performance difference can be observed when considering the stricter *mAP@50:0.95* metric, as shown in Fig. 4. The most significant improvements are observed when increasing the model size from Nano to Medium. In particular, *mAP@50:0.95* increases by approximately 12%, from 0.82 to 0.92, when moving from the Nano to the Small variant, while only a further 3% improvement is achieved by the Medium model, reaching 0.94.

As expected, increasing the model size results in larger model dimensions, as shown in Fig. 5, and higher inference times, as shown in Fig. 6 and Fig. 7. Consequently, the maximum achievable frame rate decreases as the model complexity increases.

An interesting behavior emerges when the evaluation metrics are grouped by card rank, as shown in Fig. 8 with the F1 Score. In particular, the models consistently achieve lower F1-scores for the Ace and Eight ranks, while the remaining ranks exhibit very similar, higher performance. This behavior is observed for all four suits and is consistently reproduced

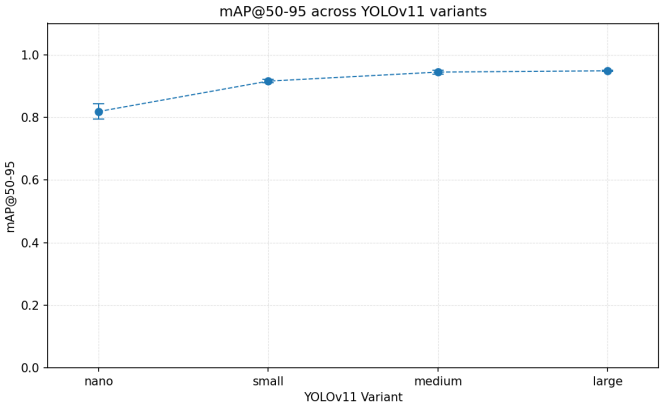


Fig. 4. mAP@50-95 across YOLOv11 variants

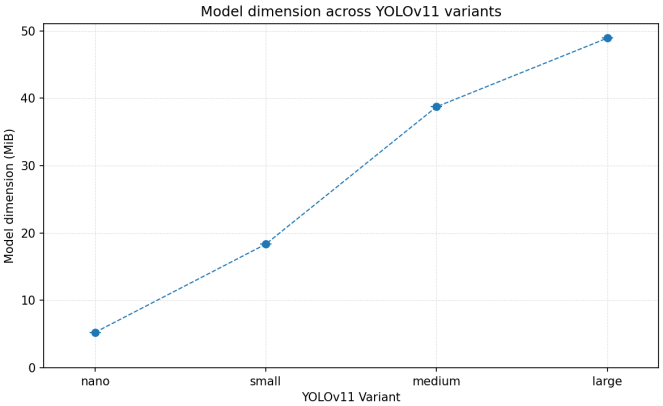


Fig. 5. Model dimension YOLOv11 variants

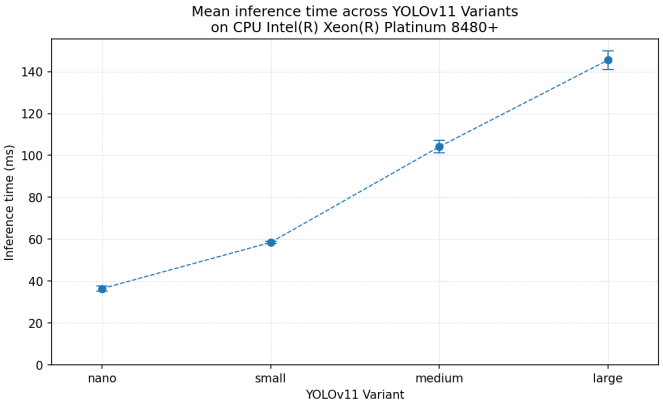


Fig. 6. Inference Time across YOLOv11 variants on CPU INTEL XEON PLATINUM 8480

across all examinated YOLOv11 variants.

A possible explanation is that this behavior originates from intrinsic characteristics of the dataset rather than from the detection model itself, given the fact that the same behavior is observed across different model variants. While the dataset is well balanced in terms of the number of samples per class, the Ace and Eight instances may present less ideal conditions,

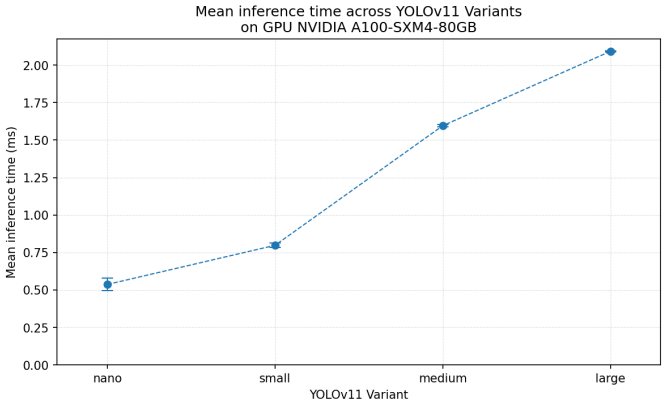


Fig. 7. Inference Time across YOLOv11 variants on GPU NVIDIA A100 SXM4 80GB

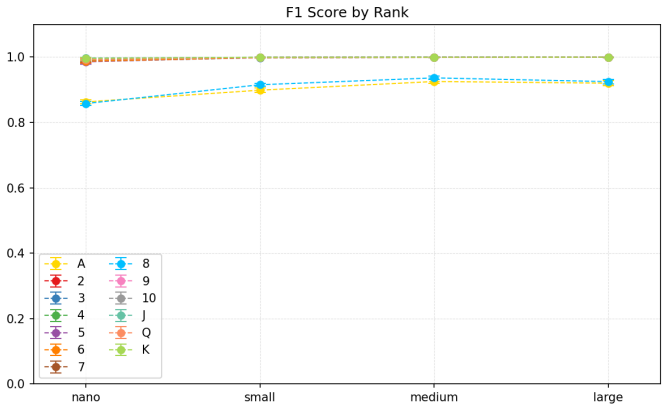


Fig. 8. F1 Score by Rank across YOLOv11 variants

such as blur, perspective distortions or illumination changes. Another possible explanation is that the graphical representation of these rank symbols provides less discriminative visual information when observed at the small scale of the card corners.

The comparison between the evaluated models highlights that increasing the model size mainly improves localization accuracy, as reflected by the *mAP@50:0.95* metric. This aspect can be relevant for real-time video-tracking applications, where stable detection across consecutive frames is essential for maintaining consistent tracking results. In such scenarios, higher localization accuracy can reduce fluctuations in predicted bounding boxes and improve the overall stability of the system.

D. Chosen model

In this work, the **Small** model was chosen. This variant represents an optimal trade-off for our final target application, as it guarantees a significant improvement in the *mAP@50-95* metric compared to the Nano model, while having only a minor increase in both model size and inference time (which rises from 40 to 60 ms on CPU).

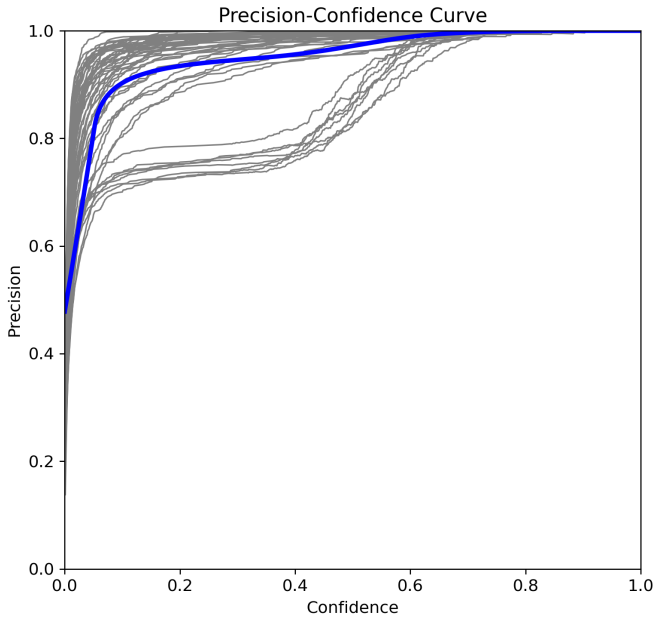


Fig. 9. Precision-Confidence curve for the model

A more detailed performance analysis of the selected model is reported through the confusion matrix in Fig. 12, the precision-recall curve in Fig. 11, and the individual precision and recall curves in Fig. 9 and Fig. 10.

As previously discussed, the classes showing performance values below the average are associated with the Ace and Eight ranks. The confusion matrix highlights that these errors are primarily related to missed detections, with some instances of these classes being classified as background, rather than being assigned to another card class. From this we can deduce that the main limitation of the selected model for these ranks is a limited ability to detect them under certain conditions, not confusion with visually similar cards.

VI. Same Card Recognition

A. Problem definition

The model we developed produces bounding boxes located on the corner of the card, over the rank and suit: this will result, for every card fully visible, in two objects of the same class (rank+suit) detected. In order to have an Intelligent System able keep track of each (real) card we want to know whether the detection of two same-class objects belong to a same underlying physical card. This is not a problem if the IS that uses our model works with a game played with a single deck of card. The problem arises whenever a game is played with two or more deck, in which cases the system must be able to distinguish how many physical cards have appeared in the frame. We call *Same Card Recognition* (***SCR***) the described problem.

B. Limitations

In order to conduct a proper analysis we are limited by the fact that the datasets don't provide ground-truth annotations

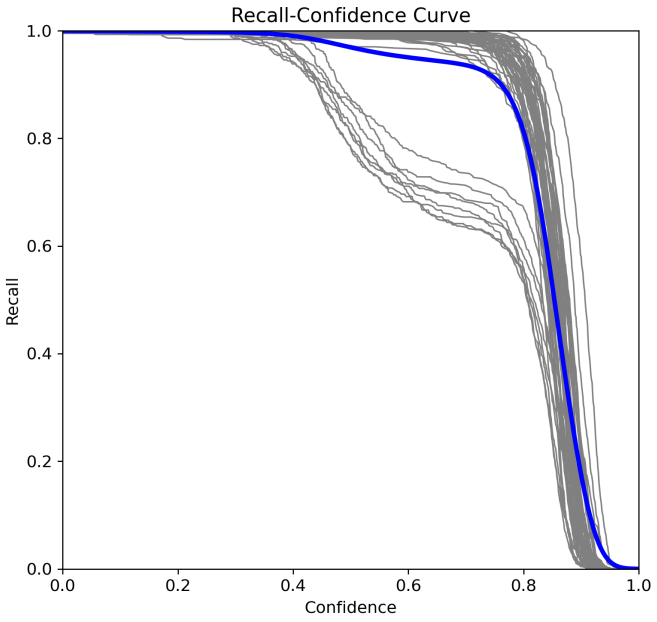


Fig. 10. Recall-Confidence curve for the model

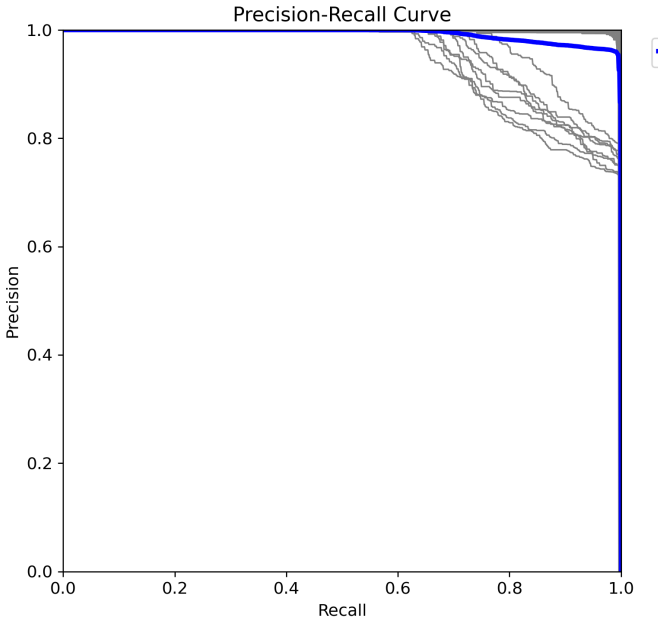


Fig. 11. Precision-Recall curve for the model

indicating whether two detected card corners belong to the same physical card or to different cards; hence we must infer this information from available data.

As an approximation we assume that when an image contains exactly two bounding boxes of the same class the two detections correspond to the two opposite corners of the same physical card; images satisfying this condition are used to construct a list of Same-Card bounding box pairs.

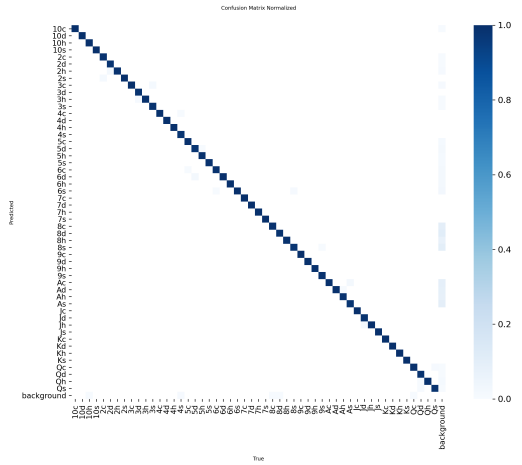


Fig. 12. Confusion matrix for the model

C. Proposed approach

Given a completely visible card the dimension of the bounding boxes and the relative position between boxes are related to the dimension of the card, and scale with the card relative size. Hence we try to derive a geometric relation between two bounding boxes of the same card, and compare it to the value of the same relation on a pair of bounding boxes belonging to different cards.

The metric proposed is the distance between two bounding boxes centers over the square root of the average of their areas, also written as ***DoA***. The *DoA* metric can be expressed as:

$$DoA(bb_1, bb_2) = \frac{\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}}{\sqrt{\frac{A_1 + A_2}{2}}} \tag{1}$$

The *DoA* metric can also be summarized as:

$$DoA(bb_1, bb_2) = \frac{d_{centers}}{\sqrt{\text{avgA}}} \tag{2}$$

Where, given a bounding box bb_i, we indicate with (x_i, y_i) the normalized coordinated of its center and with A_i its normalized area.

D. Metric analysis

Two empirical probability density functions (PDFs) were constructed, one for Same-Card pairs and one for Different-Card pairs, using Kernel Density Estimation (KDE); KDE is an alternative to normalized histogram which allows for a smooth curve for the probability density. The two empirical PDFs are shown in Fig 13.

From the Same-Card EPDF shown in Fig. 13 we can see that Same-Card bounding box pairs assume a *DoA* metric ranging from 4.5 to 6.7; Different-Card pairs on the other hand assume a more widespread *DoA* range.

In an Intelligent System based on the proposed model we can therefore evaluate the *DoA* metric over two same-class detected object; if the computed metric lies within the 4.5–6.7

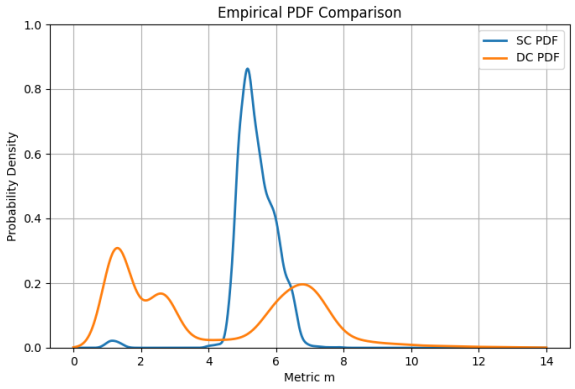


Fig. 13. Empirical PDFs for SC pairs and DC pairs

interval the two detections can be considered two corners of the same physical card.

E. Other approaches

The proposed *DoA* approach can be applied to every possible Intelligent System based our model and thus be used as a base for the SCR problem. One may develop more advanced approaches, such as video tracking of bounding boxes: with this approach one may track if the relative distance between any pairs of bounding boxes remains stable for a prolonged period of time.

VII. Intelligent System

The proposed intelligent system assists players during a Texas Hold'em poker game by combining computer vision techniques with probabilistic simulation. The overall pipeline consists of two main stages: (i) card detection through a YOLO-based object detection model and (ii) decision making based on Monte Carlo simulations. The architecture is designed to automatically recognize the current game state from one or more input images and estimate the probability of winning the hand before suggesting the most appropriate action.

A. Input Acquisition and Game State Recognition

The system requires at least one input image representing the player's private cards (hole cards). Optionally, the user may provide a second image containing the community cards (board). Depending on the detected number of community cards, the system automatically identifies the current stage of the game as follows:

- **Pre-Flop**: only the player's hole cards are available;
- **Flop**: three community cards are detected;
- **Turn**: four community cards are detected;
- **River**: five community cards are detected.

The YOLO model detects each playing card and assigns its corresponding class, encoding both rank and suit. The detected cards are then converted into a structured representation suitable for the subsequent probabilistic analysis. Since Texas Hold'em follows strict game rules, the detected configuration is also validated to ensure that duplicate cards or invalid game states are not processed.

B. Monte Carlo Hand Evaluation

After reconstructing the current game state from the detected cards, the system estimates the player's probability of winning by means of a Monte Carlo simulation. Since the identities of the remaining cards are unknown during an actual Texas Hold'em game, the algorithm approximates the outcome distribution by repeatedly generating valid random game scenarios that are consistent with the observed cards.

Let H denote the player's hole cards and B the set of currently visible community cards. The cards belonging to H and B are first removed from a standard 52-card deck, producing the set of all admissible unseen cards. At each simulation, the remaining deck is randomly shuffled and used to complete the current game configuration.

If the game has not yet reached the River stage, the missing community cards are sampled first in order to complete the board. Subsequently, two hole cards are randomly assigned to each opponent from the remaining deck, ensuring that every simulated game represents a valid and physically realizable poker scenario. This process is repeated independently for every Monte Carlo iteration.

Once the simulated game state has been completed, the system evaluates the strength of every player's hand. Since a Texas Hold'em player can form a final hand using any combination of five cards selected from the available seven cards (two private cards and five community cards), all possible five-card combinations are generated. The combination with the highest ranking according to the official Texas Hold'em rules is selected as the player's final hand.

Each five-card hand is evaluated using a deterministic ranking function that classifies the hand into one of the standard poker categories, namely Straight Flush, Four of a Kind, Full House, Flush, Straight, Three of a Kind, Two Pair, One Pair, and High Card. Besides identifying the hand category, the evaluation also computes the required tie-breaking information (e.g., highest card, kicker cards, or paired ranks), allowing two hands belonging to the same category to be compared unambiguously.

The player's best hand is then compared against the best hand of every opponent. A simulation is counted as a win if the player's hand is stronger than all opponents' hands, as a tie if at least one opponent has an equivalent hand while no stronger hand exists, and as a loss otherwise.

This procedure is repeated for a total of N independent simulations. By default, the proposed system performs

$$N = 3000$$

simulations, although this value can be configured by the user to trade computational time for estimation accuracy.

Finally, the probabilities of winning, tying, and losing are estimated through the relative frequencies observed during the simulation process:

$$P_{\text{win}} = \frac{N_{\text{win}}}{N},$$

$$P_{\text{tie}} = \frac{N_{\text{tie}}}{N},$$

$$P_{\text{lose}} = \frac{N_{\text{lose}}}{N},$$

where

$$N = N_{\text{win}} + N_{\text{tie}} + N_{\text{lose}}.$$

These probabilities provide a statistical approximation of the player's expected outcome under the assumption that the unknown cards are uniformly distributed among all valid game configurations. As the number of simulations increases, the estimated probabilities converge toward the true probabilities while requiring proportionally greater computational effort.

C. Decision Recommendation

The estimated probabilities obtained through the Monte Carlo simulation are used to generate a strategic recommendation for the player. Rather than attempting to predict the exact outcome of the current hand, the system estimates the likelihood of success under all possible completions of the game and translates this statistical information into a decision support mechanism.

The recommendation is based exclusively on the estimated winning probability P_{win}. Specifically, the system compares the computed value against two predefined decision thresholds that partition the probability space into three intervals corresponding to the actions *Fold*, *Call*, and *Raise*. Formally, the decision function can be expressed as

$$A(P_{\text{win}}) = \begin{cases} \texttt{RAISE}, & P_{\text{win}} > 0.75, \\ \texttt{CALL}, & 0.45 < P_{\text{win}} \leq 0.75, \\ \texttt{FOLD}, & P_{\text{win}} \leq 0.45. \end{cases}$$

These thresholds are **default** values, the user can change them accordingly to his preferences at the begin of the program.

According to this policy, a `RAISE` is recommended when the estimated probability of winning exceeds 75%, indicating a statistically favorable position. Conversely, a `FOLD` is suggested whenever the winning probability is below or equal to 45%, as continuing the hand would present a relatively low chance of success. Intermediate probabilities lead to a `CALL` recommendation, reflecting situations in which the hand exhibits moderate strength and remaining in the game represents a reasonable choice.

The adopted thresholds are heuristic values selected to provide intuitive recommendations and can be easily modified according to different playing styles or risk preferences. For instance, more conservative players may choose to increase the threshold required to recommend a `RAISE`, whereas more aggressive players may lower it to encourage offensive play.

It is important to emphasize that the proposed system is intended as a decision support tool rather than an autonomous poker agent. The recommendation is derived solely from the statistical strength of the current hand estimated through Monte Carlo simulation and does not incorporate contextual information such as betting history, player profiling, table position, stack sizes, pot odds, or bluffing strategies. Consequently, the suggested action should be interpreted as an objective probabilistic indication that can assist the player during decision making rather than as an optimal game-theoretic strategy.

VIII. Conclusion

In this work, the initial objectives of developing an object detection model for playing cards have been successfully achieved; furthermore, the Same-Card Recognition problem has been addressed and a basic prototype has been proposed. There are two possible ways for future development.

The first possibility is to analyze the performance of the chosen model (and the other trained variants) on a completely **natural, non-synthetic dataset**. This analysis would help determine how much the synthetic nature of the training datasets limits the model's usability for real-world images. Moreover it would allow for an assessment of the feasibility of using synthetic datasets to significantly accelerate development in similar object detection tasks. This analysis was not conducted in this study due to the lack of a suitable natural dataset and the limited time resources required to build one.

The second possibility is the completion of a **complete game analysis application**, introducing aspects such as real-time object tracking on a live video feed to dynamically monitor and analyze ongoing gameplay.

References

[1] P. Xu, R. Contant, and G. Bramley-Simmons, "Bridge Bidding via Playing Card Detection Using YOLO v3," Brown University, 2018. [Online]. Available: https://cs.brown.edu/media/filer_public/ea/f0/eaf04202-c932-4ff7-9fb2-b925fd64fe70/xuphilip.pdf

[2] T. Kostadinov, "Playing Cards Object Detection," 2024. University project, Deep Learning course, FMI. [Online]. Available: https://github.com/TeogopK/Playing-Cards-Object-Detection

[3] Tooploox, "Object Detection Simplified with YOLO and Cards," 2023. [Online]. Available: https://tooploox.com/image-detection-using-yolo-algorithm-and-poker-cards

[4] A. Singh, "Mastering Poker with AI: Real-Time Card Detection Using YOLO and Computer Vision," 2024. [Online]. Available: https://medium.com/@abhijeetas8660211/unveiling-the-cards-real-time-poker-card-detection-using-computer-vision-and-yolo-6e1c22e606ed

[5] B. Fay, L. Wong, "Poker Equity," Columbia University course project, 2023. [Online]. Available: https://www.cs.columbia.edu/~sedwards/classes/2023/4995-fall/proposals/PokerEquity.pdf

[6] Augmented Startups, "Playing Cards Dataset". [Online]. Available: https://universe.roboflow.com/augmented-startups/playing-cards-ow27d/dataset/3

[7] Andy8744, "Playing Cards Object Detection Dataset". [Online]. Available: https://www.kaggle.com/datasets/andy8744/playing-cards-object-detection-dataset